BUKTI PENGUJIAN | TESTING EVIDENCE

Greenhouse IoT Backend

Verifikasi end-to-end: sensor ingestion, kontrol perangkat via MQTT, health check, dan penanganan error — diuji langsung dengan MySQL (Laragon) dan Mosquitto (Windows Service).

Assignment Backend Engineer
PT. DMC Indonesia Food
Surabaya, Jawa Timur

RINGKASAN

Ringkasan Hasil Pengujian

Dokumen ini berisi bukti pengujian nyata (bukan simulasi) atas seluruh endpoint dan perilaku yang disyaratkan dalam assignment. Setiap pengujian dijalankan langsung melalui Swagger UI (`/docs`), diverifikasi lewat subscriber MQTT (`mosquitto_sub`) dan database client (HeidiSQL), pada lingkungan Windows dengan MySQL (Laragon) dan Mosquitto (Windows Service).

No	Skenario Pengujian	Hasil	Ref.
1	Startup backend: koneksi database & MQTT	Berhasil, tanpa error	Gambar 1
2	POST /sensor-data — data tersimpan di MySQL	201 Created, 3+ baris tersimpan	Gambar 2
3	POST /device-control — publish ke MQTT	202 Accepted, status PUBLISHED	Gambar 3
4	Verifikasi pesan MQTT diterima real-time	Pesan diterima mosquitto_sub	Gambar 4
5	Audit log tersimpan dengan command_id sama	Tercatat di tabel device_commands	Gambar 5
6	GET /status saat database dimatikan	503, database: disconnected	Gambar 6
7	POST /sensor-data saat database mati	503 DATABASE_UNAVAILABLE (bukan crash)	Gambar 7
8	Pemulihan otomatis setelah database hidup	201 Created lagi, tanpa restart server	Gambar 8

Kesimpulan: seluruh tiga endpoint wajib (POST /sensor-data, POST /device-control, GET /status) berfungsi sesuai spesifikasi, termasuk pada kondisi gagal (database/MQTT tidak tersedia). Backend tidak pernah berhenti berjalan (crash) pada seluruh skenario yang diuji.

STARTUP & KONEKSI

1. Backend Terhubung ke Database dan MQTT

Terkait requirement 2b dan 3 (koneksi database, koneksi MQTT broker)

Log berikut diambil saat backend dijalankan (`uvicorn app.main:create_app --factory --reload`). Backend berhasil membuat skema database, terhubung ke broker MQTT (Mosquitto), dan langsung subscribe ke topic sensor — semua tanpa error.

```
(.venv) C:\Users\user\Downloads\greenhouse>uvicorn app.main:create_app --factory --reload
INFO:     Will watch for changes in these directories: ['C:\\Users\\user\\Downloads\\greenhouse']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [26684] using WatchFiles
INFO:     Started server process [13780]
INFO:     Waiting for application startup.
2026-09-22T08:56:41+0700 INFO     [app.database] Database schema is ready
2026-09-22T08:56:41+0700 INFO     [app.services.mqtt_service] Connecting to MQTT broker localhost:1883 (async, auto-reconnect)
2026-09-22T08:56:41+0700 INFO     [app] Greenhouse Backend v1.0.0 started
INFO:     Application startup complete.
2026-09-22T08:56:41+0700 INFO     [app.services.mqtt_service] Connected to MQTT broker
2026-09-22T08:56:41+0700 INFO     [app.services.mqtt_service] Subscribed to greenhouse/sensor/+ (qos=1)
```

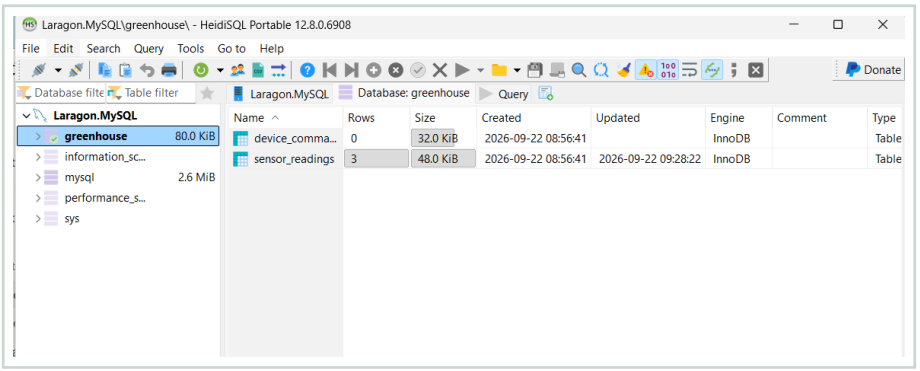
Gambar 1 — *Log startup menunjukkan* `Database schema is ready`, `Connected to MQTT broker`, *dan* `Subscribed to greenhouse/sensor/+`.

SENSOR INGESTION

2. POST /sensor-data — Data Tersimpan di Database

Terkait requirement 1a-1c (validasi, penyimpanan, response jelas)

Setelah beberapa kali memanggil `POST /sensor-data` lewat Swagger UI dengan payload valid (sensor_id, temperature, humidity), data berhasil tersimpan secara permanen di database MySQL. Tabel `sensor_readings` di bawah ini dibuka langsung lewat HeidiSQL, menunjukkan baris data hasil pengujian.



Gambar 2 — *Tabel* `sensor_readings` *di database* `greenhouse` *berisi baris data hasil pengujian POST /sensor-data, tersimpan permanen di MySQL.*

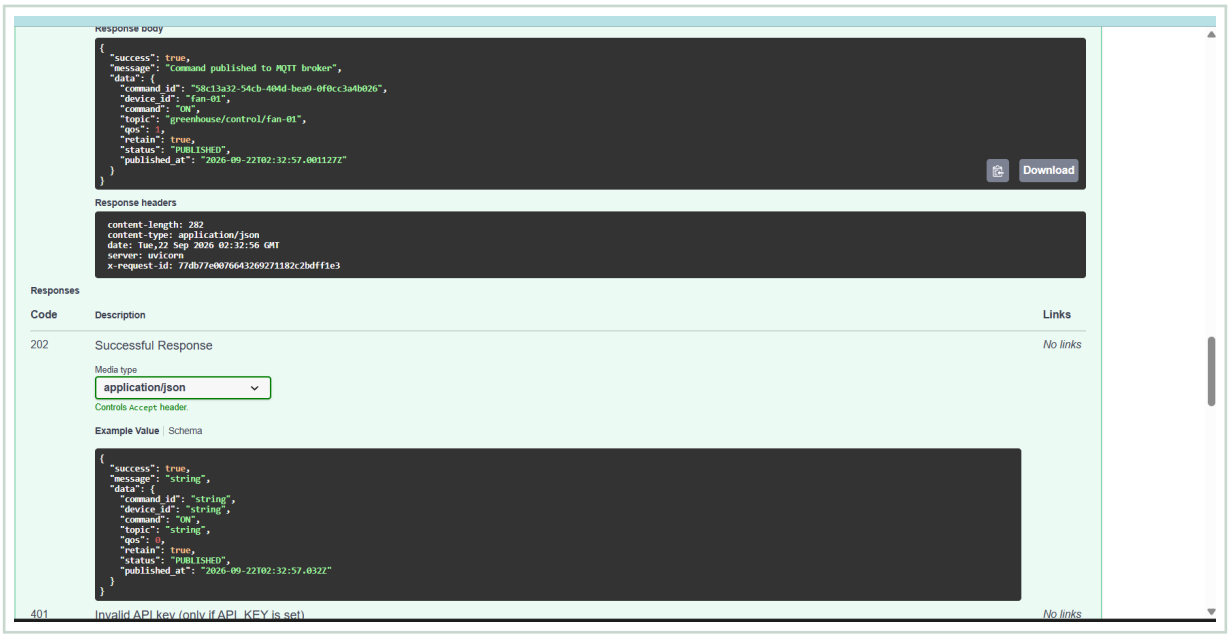
DEVICE CONTROL

3. POST /device-control — Perintah Dipublikasikan ke MQTT

Terkait requirement 2a-2c (validasi, publish MQTT, topic terstruktur)

Memanggil `POST /device-control` dengan payload `{"device_id": "fan-01", "command": "ON"}` menghasilkan response **202 Accepted**. Response menunjukkan topic MQTT yang dituju sesuai

format yang disyaratkan (`greenhouse/control/{device_id}`), beserta QoS 1 dan status `PUBLISHED`.



Gambar 3 — *Response API setelah mengirim perintah ON ke* `fan-01`*: topic* `greenhouse/control/fan-01`*, qos 1, retain true, status PUBLISHED.*

VERIFIKASI MQTT

4. Pesan Diterima Real-Time oleh Subscriber MQTT

Membuktikan publish benar-benar sampai ke broker, bukan hanya klaim di response API

Untuk memastikan perintah benar-benar dipublikasikan ke broker (bukan hanya response API yang mengklaim sukses), dijalankan subscriber independen: `mosquitto_sub -t "greenhouse/control/#" -v`. Kedua perintah (`fan-01 ON` dan `pump-02 OFF`) diterima secara real-time dengan `command_id` yang identik dengan response API.



Gambar 4 — *Terminal* `mosquitto_sub` *menerima kedua pesan kontrol secara real-time, lengkap dengan payload JSON dan command_id yang dapat ditelusuri.*

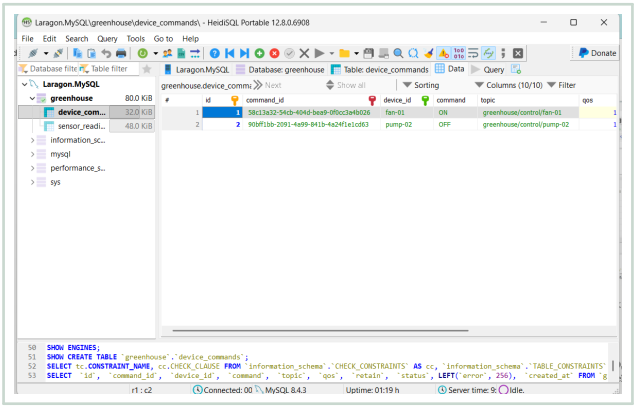
AUDIT LOG & TRACEABILITY

5. Riwayat Perintah Tercatat di Database

Fitur tambahan di luar requirement minimum, untuk traceability penuh

---

Setiap perintah kontrol juga dicatat ke tabel `device_commands` sebagai audit log. `command_id` pada tabel ini identik dengan yang muncul di response API (Gambar 3) dan di pesan MQTT (Gambar 4) — membuktikan satu perintah dapat ditelusuri secara penuh dari API, ke broker, sampai ke catatan database.



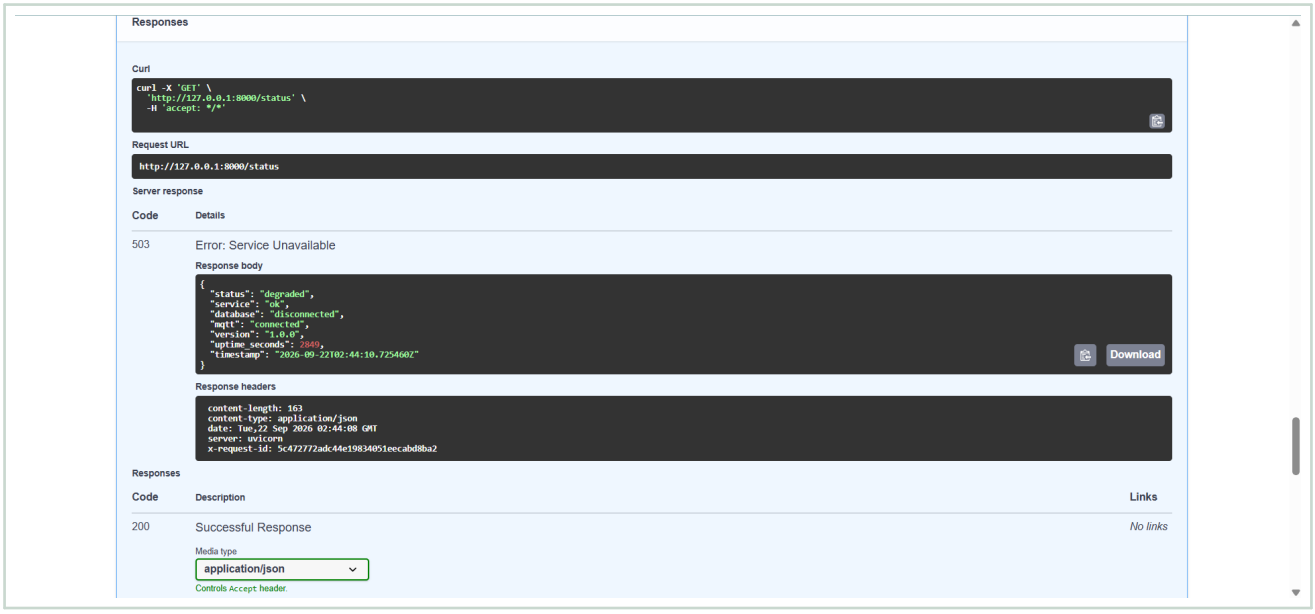
Gambar 5 — *Tabel* `device_commands` *mencatat dua perintah (fan-01/ON dan pump-02/OFF) dengan command_id yang sama persis dengan Gambar 3 dan 4.*

HEALTH CHECK — SKENARIO GAGAL

6. GET /status Saat Database Dimatikan

Terkait requirement 3a-3c (status backend, database, MQTT)

Untuk menguji perilaku pada kondisi gagal, MySQL (Laragon) dimatikan secara sengaja tanpa menghentikan backend. Endpoint `GET /status` tetap merespons (server tidak crash), dan dengan tepat melaporkan setiap komponen secara independen: database `disconnected`, sementara MQTT tetap `connected`.



Gambar 6 — *Response 503 dengan* `status: degraded`, `database: disconnected`, `mqtt: connected` — *kedua dependency dicek secara independen.*

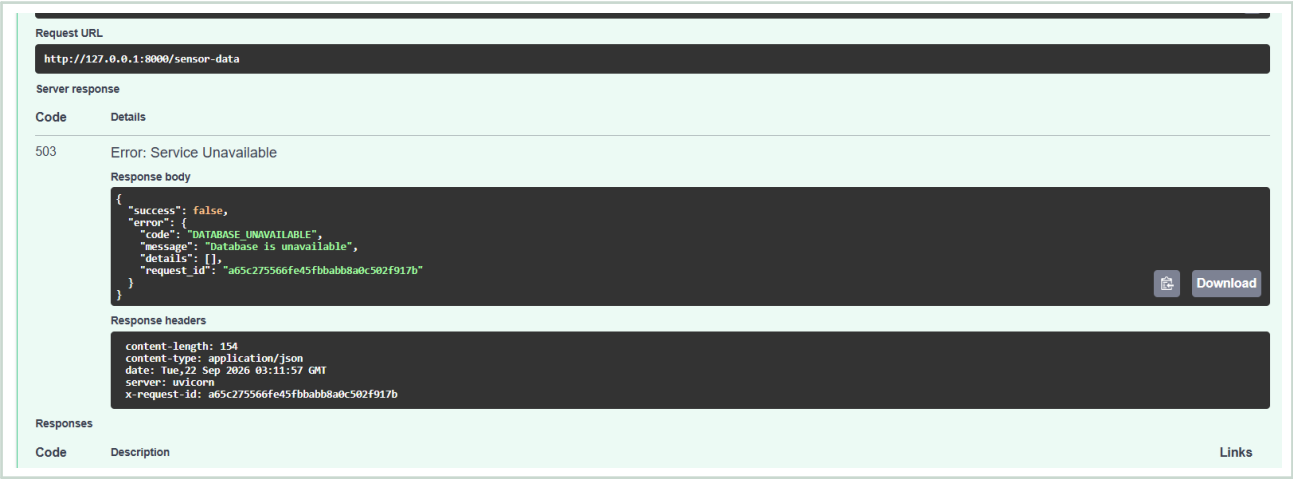
ERROR HANDLING

7. POST /sensor-data Saat Database Tidak Tersedia

Terkait requirement 1c dan bagian 'error and edge cases' dokumentasi

---

Masih dalam kondisi MySQL mati, `POST /sensor-data` dipanggil kembali. Backend mengembalikan error yang jelas dan terstruktur (`503 DATABASE_UNAVAILABLE`) alih-alih crash atau error 500 yang tidak jelas — sesuai prinsip penanganan error yang didokumentasikan di DESIGN.md.



Gambar 7 — *Response 503 dengan format error konsisten: code DATABASE_UNAVAILABLE, message jelas, dan request_id untuk keperluan tracing di log.*

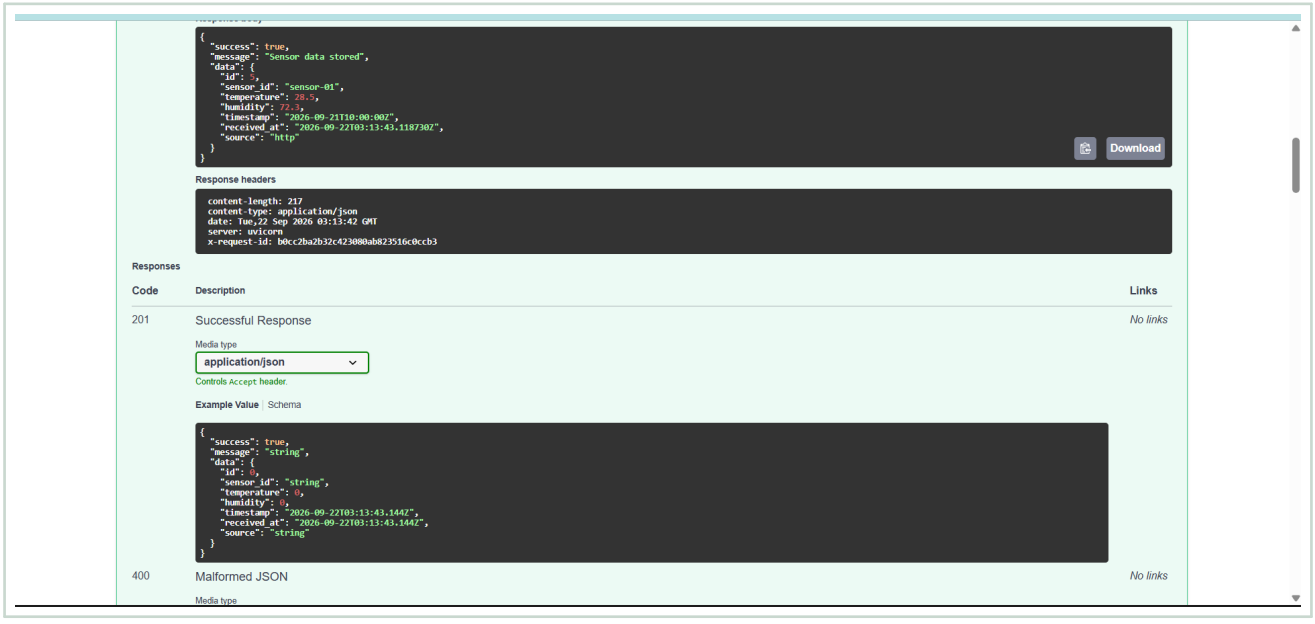
PEMULIHAN OTOMATIS

8. Sistem Pulih Sendiri Tanpa Restart Server

Membuktikan resiliency: dependency yang pulih tidak memerlukan intervensi manual

---

Setelah MySQL dinyalakan kembali, `POST /sensor-data` dipanggil sekali lagi — **tanpa merestart proses backend sama sekali**. Data berhasil tersimpan kembali (`201 Created`, id: 5), membuktikan backend mendeteksi pemulihan koneksi secara otomatis.



Gambar 8 — *Setelah MySQL kembali menyala, request langsung berhasil (201 Created) tanpa restart server — membuktikan mekanisme pemulihan otomatis bekerja sesuai desain.*

PENUTUP

Kesimpulan Pengujian

Seluruh skenario di atas dijalankan secara langsung (bukan simulasi tertulis) pada environment Windows dengan MySQL melalui Laragon dan Mosquitto sebagai Windows Service. Hasil pengujian menunjukkan:

- Ketiga endpoint wajib (sensor-data, device-control, status) berfungsi sesuai spesifikasi assignment.
- Integrasi MQTT menggunakan topic terstruktur (greenhouse/control/{device_id}), QoS 1, dan terverifikasi diterima subscriber independen.
- Data sensor tersimpan permanen di database MySQL dan dapat diverifikasi langsung.
- Setiap perintah kontrol tercatat di audit log dengan command_id yang dapat ditelusuri end-to-end.
- Backend menangani kegagalan database dengan graceful (503 terstruktur), tanpa crash.
- Backend pulih otomatis begitu dependency (database) kembali tersedia, tanpa perlu restart manual.

Dokumen pendukung lain: `README.md` (cara menjalankan), `docs/DESIGN.md` (arsitektur dan keputusan desain), `docs/presentation.pdf` (presentasi ringkas), dan source code lengkap beserta 66 automated test.